🧪 GUÍA DE LABORATORIO DE SIMULACIÓN CAPITULO 4 - Corke

🚗 Modelado y Control de Robots Móviles (Python + ROS2)

🎯 1. Objetivos

Al finalizar este laboratorio, el estudiante será capaz de:

- Comprender el modelo cinemático de robots móviles
- Implementar controladores para:
    - Ir a un punto
    - Seguimiento de línea
    - Seguimiento de trayectorias
- Analizar el comportamiento de sistemas no holonómicos
- Implementar control básico en ROS2

📘 2. Marco Teórico

En este laboratorio se estudian modelos de robots móviles basados en:

- Modelo tipo bicicleta (car-like)
- Modelo diferencial (unicycle)
- Control cinemático basado en:
    - Velocidad lineal
    - Orientación

Ecuaciones fundamentales:

$$\dot{x} = \mathrm{v}\cos\theta,$$

$$\dot{y} = \mathrm{v}\sin\theta,$$

$$\dot{\theta} = \frac{v}{L}\tan\ \delta$$

Estos modelos describen sistemas **no holonómicos**, donde existen restricciones en el movimiento lateral.

🧰 3. Requisitos

- Python 3
- Librerías:
    - numpy
    - matplotlib
- ROS2 (Humble o superior)
- Editor de código

🔹 PARTE 1: MODELO CINEMÁTICO (BICICLETA)

🧪 Actividad 1: Simulación básica

📌 Código base

```
import numpy as np
import matplotlib.pyplot as plt

L = 2.0
v = 1.0
delta = 0.3

x, y, theta = 0, 0, 0
dt = 0.1

X, Y = [], []

for _ in range(100):
    x_dot = v * np.cos(theta)
    y_dot = v * np.sin(theta)
    theta_dot = v / L * np.tan(delta)

    x += x_dot * dt
    y += y_dot * dt
    theta += theta_dot * dt

    X.append(x)
    Y.append(y)

plt.plot(X, Y)
plt.title("Trayectoria del robot")
```

```
plt.axis('equal')
plt.show()
```

❓ **Preguntas**

1. ¿Qué tipo de trayectoria describe el robot?
2. ¿Qué ocurre si δ = 0?
3. ¿Cómo afecta la longitud L al movimiento?

🔹 **PARTE 2: CONTROL – IR A UN PUNTO**

💉 **Actividad 2**

📌 **Código base**

```
import numpy as np
import matplotlib.pyplot as plt

x, y, theta = 0, 0, 0
goal = np.array([5, 5])

Kv = 0.5
Kh = 2.0

X, Y = [], []

Kv = 0.5
Kh = 1.5

dt = 0.1
L = 2.0
v = 0
delta = 0
distance =10.0

X, Y = [], []

while distance > 0.1:
    dx = goal[0] - x
    dy = goal[1] - y

    print(dx, dy)

    distance = np.sqrt(dx**2 + dy**2)
    theta_goal = np.arctan2(dy, dx)
```

```
    v = Kv * distance
    delta = Kh * (theta_goal - theta)

    x += v * np.cos(theta) * dt
    y += v * np.sin(theta) * dt
    theta += v/L * np.tan(delta) * dt

    X.append(x)
    Y.append(y)

plt.plot(X, Y)
plt.scatter(goal[0], goal[1])
plt.axis('equal')
plt.show()
```

❓ **Preguntas**

1. ¿Por qué se utiliza la función atan2?
2. ¿Qué efecto tiene el parámetro Kv?
3. ¿El robot controla orientación final? Justifique.

🔹 **PARTE 3: SEGUIMIENTO DE LÍNEA**

🍼 **Actividad 3**

📌 **Código base**

```
import numpy as np
import matplotlib.pyplot as plt

# Línea: ax + by + c = 0
a, b, c = 1, -2, 4

# Pose inicial
x, y, theta = 8, 5, np.pi/2


Kd = 0.5
Kh = 1.0

dt = 0.1
L = 2.0
v = 1
```

```
delta = 0
iteraciones = 200
n = 0

X, Y = [], []

while n < iteraciones:
    # Distancia a la línea
    distance = (a*x + b*y + c) / np.sqrt(a**2 + b**2)
    # Ángulo de la línea
    theta_line = np.arctan2(a, -b)

    #Controlador
    delta = Kd * distance + Kh * (theta_line - theta)

    # Modelo cinemático
    x += v * np.cos(theta) * dt
    y += v * np.sin(theta) * dt
    theta += v/L * np.tan(delta) * dt

    X.append(x)
    Y.append(y)
    n = n + 1

# ------------------------------
# Graficar línea homogénea
# ------------------------------
x_line = np.linspace(3, 25, 100)

if b != 0:
    y_line = (-a * x_line - c) / b
else:
    # Línea vertical
    x_line = np.full(100, -c/a)
    y_line = np.linspace(-5, 10, 100)

# ------------------------------
# Gráfica
# ------------------------------
plt.plot(X, Y, label="Trayectoria del robot")
plt.plot(x_line, y_line, 'r--', label="Línea objetivo")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Seguimiento de línea")
plt.axis('equal')
plt.grid()
plt.legend()
```

```
plt.show()
```

❓ **Preguntas**

1. ¿Qué representa la variable *distance*?
2. ¿Cuál es el rol de cada término del controlador?
3. ¿Qué ocurre si Kd = 0?

🔹 **PARTE 4: SEGUIMIENTO DE TRAYECTORIA (PURE PURSUIT)**

🧪 **Actividad 4**

📌 **Código base**

```
import numpy as np
import matplotlib.pyplot as plt

path = np.array([[0,0],[2,4],[6,5],[8,10]])
lookahead = 0.5

x, y, theta = 0, 0, 0

current_index = 0

Kd = 1.3

dt = 0.1
L = 2.0
v = 0.5
iteraciones = 285
n = 0

X, Y = [], []

def angle_wrap(a):
    return (a + np.pi) % (2*np.pi) - np.pi

while n < iteraciones:
    # Distancia al punto actual
    dx = path[current_index][0] - x
    dy = path[current_index][1] - y
    dist = np.hypot(dx, dy)

    # Avanzar al siguiente punto si está cerca
    if dist < lookahead and current_index < len(path) - 1:
```

```
        current_index += 1

    target = path[current_index]

    # Dirección al objetivo
    dx = target[0] - x
    dy = target[1] - y
    theta_goal = np.arctan2(dy, dx)

    # Error angular normalizado
    error = angle_wrap(theta_goal - theta)

    # Control
    delta = Kd * error

    # Modelo cinemático
    x += v * np.cos(theta) * dt
    y += v * np.sin(theta) * dt
    theta += v/L * np.tan(delta) * dt

    X.append(x)
    Y.append(y)
    n = n + 1

# ----------------------------
# Gráfica
# ----------------------------
plt.plot(path[:,0], path[:,1], 'r--', label="Trayectoria")
plt.plot(X, Y, label="Robot")
plt.axis('equal')
plt.grid()
plt.legend()
plt.show()
```

❓ **Preguntas**

1. ¿Qué efecto tiene el parámetro lookahead?
2. ¿Por qué el robot no sigue exactamente la trayectoria?
3. ¿Qué ocurre si el lookahead es muy pequeño o muy grande?

🔹 **PARTE 5: IMPLEMENTACIÓN EN ROS2 USANDO EL TURTLEBOT3 (CONTROL DIFERENCIAL)**

Instalar el Turtlebot3 con base en el procedimiento descrito en el link: TurtleBot3

🧪 **Actividad 5: Crear el nodo** lab_robotica/go_to_goal.py

📌 **Código base**

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TwistStamped
from nav_msgs.msg import Odometry
import numpy as np
import math

class GoToGoal(Node):

    def __init__(self):
        super().__init__('go_to_goal')

        self.pub = self.create_publisher(TwistStamped, '/cmd_vel', 10)
        self.sub = self.create_subscription(Odometry, '/odom', self.odom_callback, 10)

        self.timer = self.create_timer(0.1, self.control)

        self.x = 0
        self.y = 0
        self.theta = 0
        self.Kv = 0.5
        self.Kh = 2.0

        self.goal = np.array([2.0, 2.0])

    def odom_callback(self, msg):
        self.x = msg.pose.pose.position.x
        self.y = msg.pose.pose.position.y

        q = msg.pose.pose.orientation
        siny_cosp = 2 * (q.w*q.z + q.x*q.y)
        cosy_cosp = 1 - 2 * (q.y*q.y + q.z*q.z)
        self.theta = math.atan2(siny_cosp, cosy_cosp)

    def pose_callback(self, msg):
        self.x = msg.x
        self.y = msg.y
        self.theta = msg.theta


    def control(self):
```

```
        # ------------------------------
        # Error de posición
        # ------------------------------
        dx = self.goal[0] - self.x
        dy = self.goal[1] - self.y

        distance = np.sqrt(dx**2 + dy**2)
        angle = np.arctan2(dy, dx)

        # ------------------------------
        # Normalización angular
        # ------------------------------
        def wrap(a):
            return (a + np.pi) % (2*np.pi) - np.pi

        error = wrap(angle - self.theta)

        # ------------------------------
        # Control
        # ------------------------------
        if distance < 0.05:
            v = 0.0
            w = 0.0
        else:
            # Control proporcional
            v = self.Kv * distance
            w = self.Kh * error

        # ------------------------------
        # Mensaje TwistStamped
        # ------------------------------
        msg = TwistStamped()

        # Header (obligatorio en muchos plugins)
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = "base_link"

        # Velocidades
        msg.twist.linear.x = v
        msg.twist.angular.z = w

        self.pub.publish(msg)

def main():
    rclpy.init()
    node = GoToGoal()
    rclpy.spin(node)
    node.destroy_node()
```

```
rclpy.shutdown()
```

📌 **Terminal 1**
ros2 launch turtlebot3_gazebo empty_world.launch.py

📌 **Terminal 2**
ros2 topic echo /odom

📌 **Terminal 2**
ros2 run lab_robotica go_to_goal

Tener presente que el **tópico /odom** estima (no mide con sensores) la posición y orientación del robot integrando el modelo cinemático de un robot con control diferencial, como el Turtlebot3:

$$v = \frac{v_R + v_L}{2}$$

$$\omega = \frac{v_R - v_L}{W}$$

Y:

$$\dot{x} = v \cos\ \theta$$
$$\dot{y} = v \sin\ \theta$$
$$\dot{\theta} = \omega$$

La integración numérica:

$$x(t+dt) = x + v*\cos(\theta)*dt$$
$$y(t+dt) = y + v*\sin(\theta)*dt$$
$$\theta(t+dt) = \theta + \omega*dt$$

❓ **Preguntas**

1. ¿Qué tipo de información se publica en el tópico /cmd_vel y para que se usa?
2. ¿Qué información se utilizó para cerrar el lazo de control con la suscripción al tópico /odom?
3. ¿Qué sensores serían necesarios en un sistema real?

📦 **4. Entregables**

Cada grupo debe entregar:

📄 **Informe técnico (PDF)**

- Descripción de cada actividad
- Respuestas a las preguntas
- Gráficas obtenidas
- Análisis de resultados

💻 **Código**

- Scripts en Python
- Nodo ROS2 funcional

🎥 **(Opcional)**

- Video demostración